

Memory Hierarchy: Caches

COMS10015

Dr Tom Deakin

University of Bristol

Week 19

Cache behaviour

Important note

The hardware caches in a processor are usually a micro-architecture optimisation.

- ▶ They help the processor run faster without any input from the program.
- ▶ The ISA only sees data in the main memory space and registers.
- ▶ Means the program doesn't know where data is, that's up to the processor.
- ▶ The ISA (usually) doesn't give any controls.
- ▶ Caches help the performance of the code without any programmer intervention.

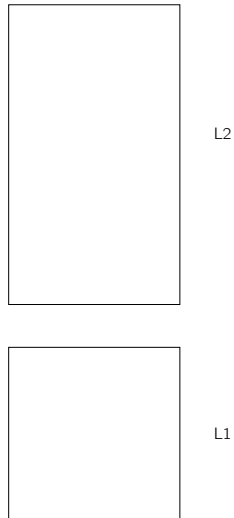
This is usually a good thing, but part of my research involves finding and understanding data access patterns which are unfriendly to caches.

Inclusivity

- ▶ Processors can have inclusive and/or exclusive caches:
 - Inclusive Any data in a lower cache level is also present in this cache.
 - Exclusive Data in lower cache levels is not necessarily in this cache.
- ▶ Two common schemes today:
 - ▶ All the data caches are inclusive: $L1 \subset L2 \subset L3$.
 - ▶ L2 is inclusive of L1. L3 is exclusive and just* contains things evicted from L2.

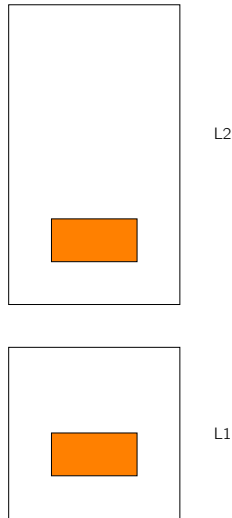
Example: Cache inclusivity

- ▶ Take two levels of cache: L1 and L2.
- ▶ L2 is *inclusive* of L1.



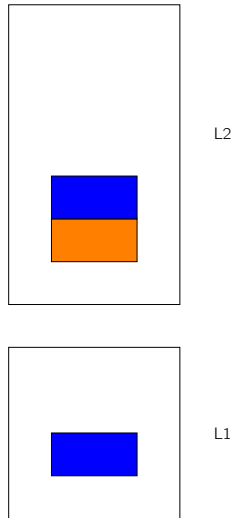
Example: Cache inclusivity

- ▶ Take two levels of cache: L1 and L2.
- ▶ L2 is *inclusive* of L1.
- ▶ When the processor loads data into L1 cache, it must also put it in L2.



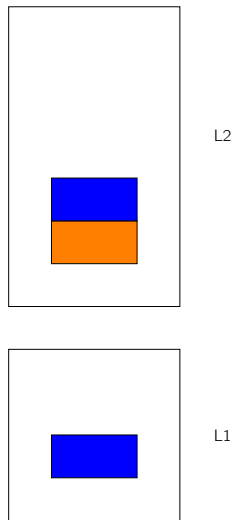
Example: Cache inclusivity

- ▶ Take two levels of cache: L1 and L2.
- ▶ L2 is *inclusive* of L1.
- ▶ When the processor loads data into L1 cache, it must also put it in L2.
- ▶ When the data leaves L1 (evicted), it might still be in L2.



Example: Cache inclusivity

- ▶ Take two levels of cache: L1 and L2.
- ▶ L2 is *inclusive* of L1.
- ▶ When the processor loads data into L1 cache, it must also put it in L2.
- ▶ When the data leaves L1 (evicted), it might still be in L2.
- ▶ Future access to the data means loading only from L2 instead of main memory.



ISA view of memory

Recall: ISA addressing modes.

- ▶ Instructions access data in *registers* or *memory*.
- ▶ The ISA sees a unified, flat memory space, with no concept of cache location.
- ▶ E.g. Arm load instruction: `LDR R0, [R1]` or `LDR R0, 0x04`

When memory access instruction executed, caches are checked, and loaded from memory if not found.

Hits and Misses

Definition: Cache hits and misses

A *cache hit* occurs when the requested data **is found** in the cache.

A *cache miss* occurs when the requested data **it not found** in the cache.

- ▶ If cache miss in lower level of cache, next level is checked.
- ▶ Eventually, memory request might be satisfied from main memory.
- ▶ Data copied into one or more levels of cache, ready for subsequent accesses.

Locality

Working set

Definition: Working set

The working set of a program is the amount of memory it requires during a time period.

Caches help keep the current working set close to the data path to improve performance.

Cache blocks (1)

Memory logically divided into blocks of a fixed size.

- ▶ Often called cache lines.
- ▶ Data is transferred between caches and memory in blocks.
- ▶ Common size is 64-bytes.

Data moves through cache hierarchy in cache lines, rather than as individual words.

Cache blocks (2)

Moving data in blocks:

- ▶ Little additional cost to transfer additional words.
- ▶ Already paid the latency cost for memory access with first word.
- ▶ High probability the rest of block will be needed anyway.

Locality

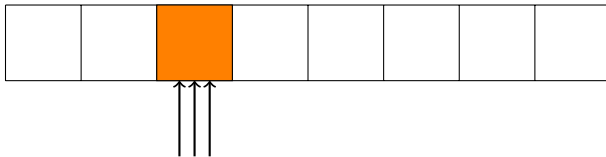
- ▶ Locality means nearby things are related.
- ▶ Close in time: if used something, likely to use it again soon.
- ▶ Close in space: if used something, something nearby likely to be used.

Caches work by taking advantage of locality.

Temporal locality

Definition: Temporal locality

The same memory location is accessed repeatedly in a short period of time.

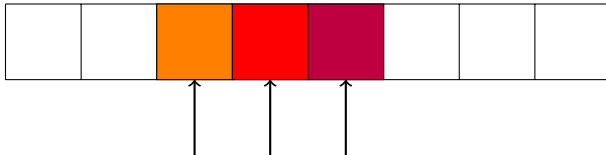


- ▶ Very likely need to access the same data again soon.
- ▶ Programs often have small working set.
- ▶ Access the data in the working set a lot during that time period.
- ▶ Caches exploit temporal locality by keeping copy of recently accessed data.

Spatial locality

Definition: Spatial locality

Nearby memory locations which similar addresses are accessed in succession in a short period of time.



- ▶ Very likely to need to access adjacent data.
- ▶ E.g. Arrays and data structures stored in nearby memory.
- ▶ Caches exploit spatial locality by moving data in cache lines/blocks of data.

Direct mapped cache

Putting blocks in cache (1)

Where do you put each memory block in cache?

- ▶ Similar idea to hash table data structure:
 - ▶ Need to map larger set to smaller set.
 - ▶ Use a hash function to map memory address into blocks/lines in cache.
- ▶ Use bits of memory address to determine location:
 - ▶ Lowest bits for word in block/line.
 - ▶ Next bits for location in the cache.
- ▶ Equivalent to modulo (%) operations on memory address.

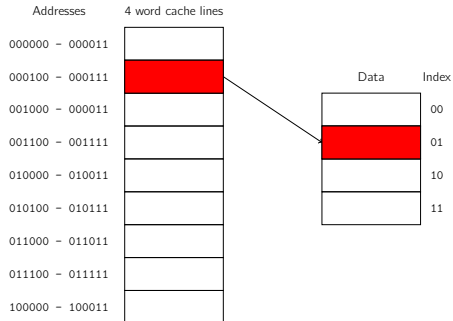
Putting blocks in cache (2)

Addresses	4 word cache lines
000000 - 000011	
000100 - 000111	
001000 - 000011	
001100 - 001111	
010000 - 010011	
010100 - 010111	
011000 - 011011	
011100 - 011111	
100000 - 100011	

Data	Index
	00
	01
	10
	11

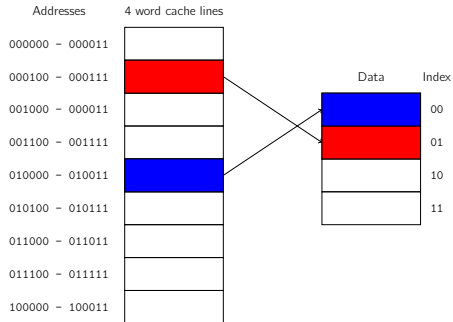
- ▶ E.g. Cache can hold four blocks of four words each.
- ▶ Lowest 2 bits for word in line/block.
- ▶ Next bits for location in the cache.

Putting blocks in cache (2)



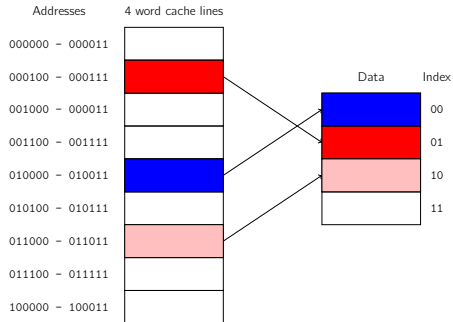
- ▶ E.g. Cache can hold four blocks of four words each.
- ▶ Lowest 2 bits for word in line/block.
- ▶ Next bits for location in the cache.
 - ▶ Take address 00 01 00, $1\%4 = 1$.

Putting blocks in cache (2)



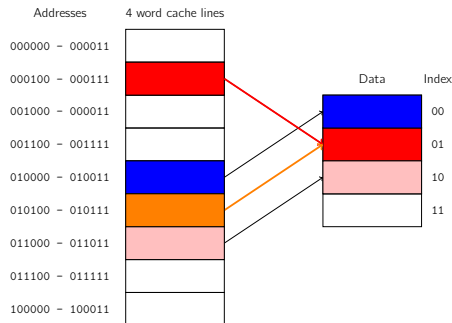
- ▶ E.g. Cache can hold four blocks of four words each.
- ▶ Lowest 2 bits for word in line/block.
- ▶ Next bits for location in the cache.
 - ▶ Take address 00 01 00, $1\%4 = 1$.
 - ▶ Address 01 00 00, $4\%4 = 0$.

Putting blocks in cache (2)



- ▶ E.g. Cache can hold four blocks of four words each.
- ▶ Lowest 2 bits for word in line/block.
- ▶ Next bits for location in the cache.
 - ▶ Take address 00 01 00, $1\%4 = 1$.
 - ▶ Address 01 00 00, $4\%4 = 0$.
 - ▶ Address 01 10 00, $6\%4 = 2$.

Putting blocks in cache (2)



But could have conflicts!

- ▶ E.g. Cache can hold four blocks of four words each.
- ▶ Lowest 2 bits for word in line/block.
- ▶ Next bits for location in the cache.
 - ▶ Take address 00 01 00, $1\%4 = 1$.
 - ▶ Address 01 00 00, $4\%4 = 0$.
 - ▶ Address 01 10 00, $6\%4 = 2$.
 - ▶ Address 01 01 00, $5\%4 = 1$.

Direct mapped cache (1)

Definition: Direct mapped cache

Each memory address is mapped to one (and only one) location in cache.

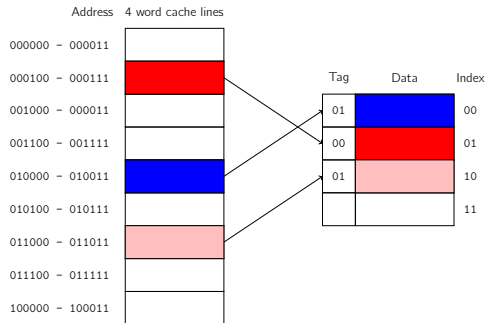
- ▶ Previous slide shows a direct mapped cache.
- ▶ Modulo function used to determine location in cache.
 - ▶ Equivalent to selecting bits of memory address.
- ▶ Lowest bits determine location within a cache line/block.
- ▶ Higher bits determine location within the cache.
- ▶ Number of bits required determined by number of blocks in cache.
- ▶ E.g. 32 KB (2^{15} bytes) cache has 512 blocks (with 64-byte cache lines): 9 bits required to select location plus 3 bits to select 64-bit words from line.

Direct mapped cache (2)

- ▶ In general, conflicts would evict the previous block and store the latest cache line.
- ▶ But, with multiple addresses mapping to same cache block, how do we distinguish them?

Tag store

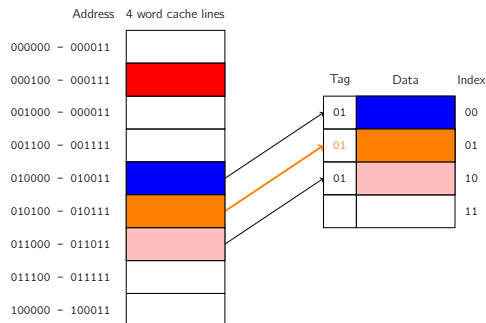
Tag Store keeps records of which addresses are in cache.



- ▶ Lower bits determine position in cache.
- ▶ Remaining higher bits saved in Tag Store.
- ▶ Combine tag and cache index to reconstruct original address.
- ▶ E.g. Data in Index 10 with Tag 01 and is addresses 011000 - 011011.

Tag store

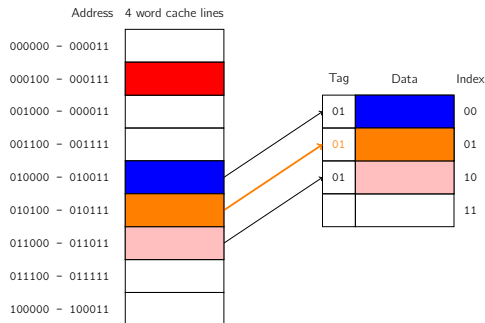
Tag Store keeps records of which addresses are in cache.



- ▶ Lower bits determine position in cache.
- ▶ Remaining higher bits saved in Tag Store.
- ▶ Combine tag and cache index to reconstruct original address.
- ▶ E.g. Data in Index 10 with Tag 01 and is addresses 011000 - 011011.
- ▶ When data is replaced, tag is also updated.

Tag store

Tag Store keeps records of which addresses are in cache.



- ▶ Lower bits determine position in cache.
- ▶ Remaining higher bits saved in Tag Store.
- ▶ Combine tag and cache index to reconstruct original address.
- ▶ E.g. Data in Index 10 with Tag 01 and is addresses 011000 - 011011.
- ▶ When data is replaced, tag is also updated.

Know what is in cache by checking the tag store.

Meta data: Valid and Modify bits

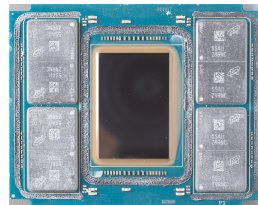
Caches also contain additional bits to keep track of other information.

- ▶ Valid bit: is the cache line valid data (i.e. used)?
- ▶ Modify bit: is the cache line different to main memory?

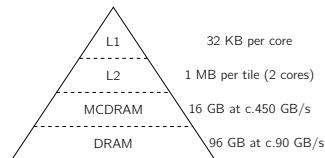
Used to make sure the memory hierarchy is consistent.

Example: Intel Xeon Phi (Knights Landing)

- ▶ Has both DDR DRAM and 16 GB MCDRAM.
- ▶ MCDRAM can be configured as a Last Level direct mapped cache.
 - ▶ 64-byte cache lines (just like L1 and L2).
- ▶ DDR is $6\times$ capacity of MCDRAM.
 - ▶ 6 addresses map to the same cache line.
- ▶ MCDRAM has more bandwidth but higher latency than DDR.
 - ▶ Contrast to on-chip caches which have lower latency.
- ▶ Cache improves performance of programs with high locality without changing program source code.



Picture from Wikipedia CC0



Further Reading

- ▶ Cache inclusion policy:
https://en.wikipedia.org/wiki/Cache_inclusion_policy